

Assignment 3:

Date: 20/8/2026

1. Give answers to the following:

Consider: Employee IDs: 101, 102, 103, 104, 105

Solve the following using different collections:

- A. Store IDs where order matters.
- B. Store IDs where duplicates should automatically disappear.
- C. Store: Employee ID → Employee Name
- D. Store a fixed employee record: Employee ID, Name, Department

For each solution answer:

- a. Which collection did you choose?
- b. Why?
- c. What operation becomes easy?
- d. What operation becomes difficult?
- e. Is the collection mutable or immutable?

2. Find and correct the Mistakes:

The following code contains conceptual mistakes. Identify the mistakes and rewrite the code correctly. Explain why this creates a problem and identify which collection could solve it.

Code 1

```
student = {  
    ["name", "Rahul"],  
    ["age", 22]  
}
```

Code 2

```
permissions = {  
    "read",  
    "write",  
    "delete"  
}  
permissions["execute"] = True
```

Code 3

```
numbers = {1, 2, 3}
numbers.append(4)
```

Code 4

```
employee = (101, "Rahul", "IT")
employee[1] = "Raj"
```

Code 5

```
data = {
    [1, 2, 3]: "Python"
}
```

Code 6

remove() vs discard() in Set
O/P for :

i) numbers = {1, 2, 3}
numbers.remove(5)

ii) numbers = {1, 2, 3}
numbers.remove(5)

Code 7 Trace and Predict the o/p

```
import copy
records = {"batch": [[1, 2], [3, 4]]}
```

A. snapshot_a = records
snapshot_b = records.copy()
snapshot_c = copy.deepcopy(records)

B. records["batch"][0].append(99)
records["batch"] = records["batch"] + [[5, 6]]

```
C. print(snapshot_a["batch"])
    print(snapshot_b["batch"])
    print(snapshot_c["batch"])
```

3. What is the purpose of: `dict.get()` -> Instead of directly accessing a key?

4. Choose the Correct Data Structure

For each problem, select the most appropriate Python collection and explain why.

Case 1

You need to store: 10, 20, 30, 40, 50

The order must be preserved, and duplicate values are allowed.

Case 2

You need to store:

- Employee ID
- Employee Name
- Joining Date

The record should not accidentally be modified.

Case 3

You need to find an employee immediately using: Employee ID → Employee Details

Case 4

You need to store unique email addresses.

Duplicate emails must automatically be eliminated.

Case 5

You have a fixed set of application permissions that should never be modified.

Case 6

You need to store: Department → Employees,
Each department can contain many employees.
Which collection combination would you use?

Case 7

You need to find common users between two online groups.
Example:

Group A = {A, B, C, D}

Group B = {C, D, E, F}

Which operation would you use?

5. Hospital Patient Management System

Develop a Python-based hospital patient management system.

The system should maintain information about multiple patients.

For each patient, the system should be capable of maintaining:

Can do through static input values

- A unique patient identifier
- Patient Information
 - Patient name
 - Age
 - Blood group
 - One or more diseases/medical conditions
 - One or more contact numbers
 - Emergency contact information

The system should support the following operations:

1. Add a new patient.
2. Search for a patient using the patient's unique identifier.
3. Update patient information.
4. Add a new disease/medical condition for a patient.
5. Remove a disease/medical condition.

6. Display all the different diseases recorded in the hospital.
7. Find all patients having a particular disease.
8. Find patients belonging to a particular blood group.
9. Maintain the list of valid blood-group categories so that these predefined categories cannot be accidentally modified.
10. Allow a patient to have multiple contact numbers.
11. Display complete information of all patients.
12. Delete a patient.
13. Find the patient having the highest number of recorded diseases.

Design Challenge

You must decide for yourself which Python collection should be used for:

- Storing all patients
- Representing an individual patient's information
- Storing multiple diseases
- Storing multiple contact numbers
- Representing fixed information
- Maintaining predefined categories

* Select the collection based on the nature of the data and the operations required.

* Do not copy a predefined data-structure mapping. The choice must be yours.

6. Movie Ticket Booking System

The system should maintain:

- Multiple movies
- Multiple timings for each movie
- Customers
- Customer bookings
- Number of seats/tickets
- Booking information

The program should provide the following operations:

1. Add a movie.
2. Add timings for a movie.
3. Add a customer booking.
4. Search bookings using the Booking ID.

5. Search all bookings made by a customer.
6. Update the number of tickets.
7. Cancel a booking.
8. Display all unique movies.
9. Display all unique customers.
10. Find common movie timings between two movies.
11. Display the movie with the highest number of bookings.
12. Calculate total tickets sold.
13. Display all bookings in a readable format.

- You must use **at least three different Python collection types**.do **not select the collections randomly**.